

Notebook

February 13, 2025

1 Theoretische und praktische Anwendung der bedingten Wahrscheinlichkeiten

1.1 Aufgabe 3:

Nach der jährlichen Kontrolluntersuchung hat der Arzt eine gute und eine schlechte Nachricht. Die schlechte Nachricht ist, dass Sie positiv auf eine schwere Krankheit getestet wurden, und der Test zu 99 % genau sei (d. h. die Wahrscheinlichkeit, dass der Test positiv ist, wenn Sie die Krankheit haben, beträgt 0,99, ebenso wie die Wahrscheinlichkeit, dass er negativ ist, wenn Sie die Krankheit nicht haben).

1.1.1 Situation:

Aus der Aufgabenstellung heraus lassen sich verschiedene Wahrscheinlichkeiten ablesen. Dazu zählen:

- **Wahrscheinlichkeit, dass der Test positiv ist, wenn die Krankheit da ist:**
 $P(\text{Test} \mid \text{Krank}) = 99\%$
- **Wahrscheinlichkeit, dass der Test negativ ist, wenn die Krankheit nicht da ist:**
 $P(\neg \text{Test} \mid \neg \text{Krank}) = 99\%$
- **Wahrscheinlichkeit, dass der Test positiv ist, wenn die Krankheit nicht da ist:**
 $P(\text{Test} \mid \neg \text{Krank}) = 1\%$
- **Wahrscheinlichkeit, dass der Test negativ ist, wenn die Krankheit da ist:**
 $P(\neg \text{Test} \mid \text{Krank}) = 1\%$

1.1.2 Augabenteil a)

Simulieren Sie das Ergebnis für den Fall dass lediglich eine von 10.000 Personen in ihrem Alter die Krankheit betrifft und für den Fall, dass die Krankheit eine von 100 Menschen in Ihrem Alter betrifft (also $P(\text{Krank})=0.0001$ bzw. 0.01%)

1.1.3 Lösung:

Es wird ein Programm vorgestellt, welches die Testgenauigkeiten der einzelnen Fälle analysiert. Dazu werden zwei Simualtionen, einmal mit einer Krankheitswahrscheinlichkeit von $P(\text{Krank}) = 1\%$ und einer Wahrscheinlichkeit von $P(\text{Krank}) = 0.01\%$.

Zuerst importieren wir wieder die Bibliotheken `random` und `math`.

```
[425]: import random
import math
```

Danach werden die verschiedensten Szenarien dokumentiert und gezählt

```
[426]: sick_people : int = 0
healty_people : int = 0
positive_tests : int = 0
negative_tests : int = 0

sick_and_positive : int = 0
sick_and_negative : int = 0
healty_and_positive : int = 0
healty_and_negative : int = 0
```

Die Wahrscheinlichkeit, dass ein Test das falsche Ergebniss ausgibt, also negativ bei krank, oder positiv bei gesund, liegt stets bei 1%. Des weiteren werden die boolschen Variablen `is_sick` und `is_positive` eingeführt.

```
[427]: probability_false_test_statement : float = 0.01

is_sick : bool
is_positive : bool
```

Der Simulationsprozess funktioniert nun wie folgt:

Die Funktion `simulate` erwartet eine Krankheitswahrscheinlichkeit, für den ersten Fall 0.01% und für den zweiten Fall 1%. Dabei iterieren wir über eine Stichprobe von 100000 Personen.

Zuerst wird eine kranke, oder gesunde Person generiert. Ist die Person krank, generieren wir nun einen positiven, oder negativen Test, basierend auf den Zustand der Person. Bei krank, ist die Chance einen positiven Test zu erlangen 99%. Allgemein erhöht sich hier der Zähler für kranke Personen um 1 und die Person wird als krank deklariert.

Nun wird das Testergebniss getestet. ist es negativ, so erhöht sich der Zähler der negativen Tests um 1 und die Person wird als negativ getestet deklariert. Andernfalls ist das Testergebniss positiv.

Das selbe gilt nun für den Fall, dass die Person gesund ist.

Zuletzt werden dann die Daten geprüft.

- Ist eine Person krank und positiv getestet, so wird der Zähler `sick_and_positive` erhöht.
- Ist eine Person krank und negativ getestet, so wird der Zähler `sick_and_negative` erhöht.
- Ist eine Person gesund und positiv getestet, so wird der Zähler `healty_and_positive` erhöht.
- Ist eine Person gesund und negativ getestet, so wird der Zähler `healty_and_negative` erhöht.

Zuletzt werden dann die relativen Häufigkeiten der einzelnen Auftreten der verschiedenen Fälle ausgegeben.

```
[428]: def simulate(probability_sick : float, iterations : int = 100000):
    global sick_people
    global healty_people
```

```

global positive_tests
global negative_tests

global sick_and_positive
global sick_and_negative
global healthy_and_positive
global healthy_and_negative

for _ in range(iterations):
    generate_sick_not_sick : float = random.random()

    if generate_sick_not_sick < probability_sick:
        generate_test_positive_not_positive : float = random.random()
        sick_people += 1
        is_sick = True

        if generate_test_positive_not_positive <
↪probability_false_test_statement:
            negative_tests += 1
            is_positive = False

        else:
            positive_tests += 1
            is_positive = True

    else:
        generate_test_positive_not_positive : float = random.random()
        healthy_people += 1
        is_sick = False

        if generate_test_positive_not_positive <
↪probability_false_test_statement:
            positive_tests += 1
            is_positive = True

        else:
            negative_tests += 1
            is_positive = False

    if is_sick:
        if is_positive:
            sick_and_positive += 1
        else:
            sick_and_negative += 1

    else:
        if is_positive:

```

```

        healty_and_positive += 1
    else:
        healty_and_negative += 1

    test_scenarios = ["kranke Personen", "gesunde Personen", "positiv getestete_
↳Personen",
                    "negativ getestete Personen", "positiv getestete kranke_
↳Personen",
                    "negativ getestete kranke Personen", "positiv getestete_
↳gesunde Personen",
                    "negativ getestete gesunde Personen"]

    test_cases = [sick_people, healty_people, positive_tests, negative_tests,
                  sick_and_positive, sick_and_negative, healty_and_positive,
                  healty_and_negative]

    for i in range(len(test_cases)):
        relative_frequency = math.floor(test_cases[i] / iterations * 100000) /
↳1000
        print(f"Der Testfall: \"{test_scenarios[i]}\" hatte die_
↳Wahrscheinlichkeit: {relative_frequency}%")

```

Des Weiteren wollen wir nach der Simulation und finalen Auswertung der Ergebnisse die Simulationsvariablen wieder auf Null setzen, um sie wiederzuverwenden. Dies übernimmt dabei die Funktion `reset_simulation_values`.

```

[429]: def reset_simulation_values():
        global sick_people
        global healty_people
        global positive_tests
        global negative_tests

        global sick_and_positive
        global sick_and_negative
        global healty_and_positive
        global healty_and_negative

        sick_people = 0
        healty_people = 0
        positive_tests = 0
        negative_tests = 0

        sick_and_positive = 0
        sick_and_negative = 0
        healty_and_positive = 0
        healty_and_negative = 0

```

Eine interessantere Betrachtung der Simulation geben die Testgenauigkeiten der Fälle. Diese kön-

nen in die Kategorien:

- **True positive:** Für richtig erfasste positive Tests $P(Krank | Test)$,
- **False positive:** Für falsch erfasste negative Tests $P(\neg Krank | Test)$,
- **True negative:** Für richtig erfasste negative Tests $P(Krank | \neg Test)$ und
- **False negative:** Für falsch erfasste negative Tests $P(\neg Krank | \neg Test)$.

Die Funktion `calculate_test_precision` berechnet dabei wie genau die Tests sind. Dazu teilt man für die jeweilige Situation immer die Gesamtzahl der richtigen Fälle geteilt durch die Gesamtzahl aller Fälle. Z.B kann so der Fall **True positive** über die Anzahl aller positiv getesteten kranken Person, geteilt durch die Anzahl der positiv getesteten Personen berechnet werden.

```
[430]: def test_precision():
    true_positive : float = sick_and_positive / positive_tests
    false_positive : float = healthy_and_positive / positive_tests

    true_negative : float = healthy_and_negative / negative_tests
    false_negative : float = sick_and_negative / negative_tests

    test_precision : list = [true_positive, false_positive, true_negative,
↪false_negative]

    return test_precision
```

Nun simulieren wir den Fall, dass die Krankheitswahrscheinlichkeit bei $P(Krank) = 0.01\%$ liegt und geben die Testgenauigkeit aus.

```
[431]: precision_cases : list = ["richtig positiv getestete Personen", "falsch positiv_
↪getestete Personen",
                                "richtig negativ getestete Personen", "falsch negativ_
↪getestete Personen"]

simulate(0.0001)
precision_values : list = test_precision()
reset_simulation_values()

for i in range(len(precision_values)):
    if i == 0:
        print("\n")
        print(f"Die Testgenauigkeit für das Ereigniss \"{precision_cases[i]}\":
↪liegt bei {math.floor(precision_values[i] * 100000) / 1000}%")
```

Der Testfall: "kranke Personen" hatte die Wahrscheinlichkeit: 0.019%

Der Testfall: "gesunde Personen" hatte die Wahrscheinlichkeit: 99.981%

Der Testfall: "positiv getestete Personen" hatte die Wahrscheinlichkeit: 1.001%

Der Testfall: "negativ getestete Personen" hatte die Wahrscheinlichkeit: 98.999%

Der Testfall: "positiv getestete kranke Personen" hatte die Wahrscheinlichkeit: 0.019%

Der Testfall: "negativ getestete kranke Personen" hatte die Wahrscheinlichkeit:

0.0%

Der Testfall: "positiv getestete gesunde Personen" hatte die Wahrscheinlichkeit: 0.982%

Der Testfall: "negativ getestete gesunde Personen" hatte die Wahrscheinlichkeit: 98.999%

Die Testgenauigkeit für das Ereigniss "richtig positiv getestete Personen" liegt bei 1.898%

Die Testgenauigkeit für das Ereigniss "falsch positiv getestete Personen" liegt bei 98.101%

Die Testgenauigkeit für das Ereigniss "richtig negativ getestete Personen" liegt bei 100.0%

Die Testgenauigkeit für das Ereigniss "falsch negativ getestete Personen" liegt bei 0.0%

Was hier auffällt, ist, dass die Wahrscheinlichkeit, dass eine Person richtig positiv auf die Krankheit getestet wird, nur bei ca. 1% liegt. Jedoch sieht man auch, wenn man die weiteren Werte betrachtet, dass so gut wie alle negativen Tests, richtig negativ sind. Das bedeutet, auch wenn die Anzahl der richtig positiven Tests gering ist, werden alle Personen, die die Krankheit haben, positiv auf die Krankheit getestet. Jedoch werden viele Personen, die die Krankheit nicht haben, unsicher gemacht, ob sie die Krankheit haben oder nicht.

Als nächstes wird der Testfall für $P(Krank) = 1\%$ simuliert.

```
[432]: simulate(0.01)
precision_values : list = test_precision()
reset_simulation_values()

for i in range(len(precision_values)):
    if i == 0:
        print("\n")
        print(f"Die Testgenauigkeit für das Ereigniss \"{precision_cases[i]}\"  

        ↳liegt bei {math.floor(precision_values[i] * 100000) / 1000}%")
```

Der Testfall: "kranke Personen" hatte die Wahrscheinlichkeit: 1.047%

Der Testfall: "gesunde Personen" hatte die Wahrscheinlichkeit: 98.953%

Der Testfall: "positiv getestete Personen" hatte die Wahrscheinlichkeit: 2.074%

Der Testfall: "negativ getestete Personen" hatte die Wahrscheinlichkeit: 97.926%

Der Testfall: "positiv getestete kranke Personen" hatte die Wahrscheinlichkeit: 1.037%

Der Testfall: "negativ getestete kranke Personen" hatte die Wahrscheinlichkeit: 0.01%

Der Testfall: "positiv getestete gesunde Personen" hatte die Wahrscheinlichkeit: 1.037%

Der Testfall: "negativ getestete gesunde Personen" hatte die Wahrscheinlichkeit: 97.916%

Die Testgenauigkeit für das Ereigniss "richtig positiv getestete Personen" liegt bei 50.0%

Die Testgenauigkeit für das Ereigniss "falsch positiv getestete Personen" liegt bei 50.0%

Die Testgenauigkeit für das Ereigniss "richtig negativ getestete Personen" liegt bei 99.989%

Die Testgenauigkeit für das Ereigniss "falsch negativ getestete Personen" liegt bei 0.01%

Was hier auffällt ist, dass die Wahrscheinlichkeit auf richtig positive Tests stark ansteigt, auf etwa 50%. Jedoch steigt die Wahrscheinlichkeit, dass eine Person, die die Krankheit hat, als negativ eingestuft wird, und zwar von vorhin ca. 0% auf 0.01%.

1.1.4 Aufgabenteil b)

Wie groß ist die Wahrscheinlichkeit $P(Krank | Test)$, dass Sie die Krankheit tatsächlich haben in beiden Fällen?

Mathematische Lösung: Um die Wahrscheinlichkeit zu berechnen, dass eine Person krank ist, unter der Voraussetzung, dass ihr Test positiv ist, kann erneut der Satz von Bayes verwendet werden. Dieser besagt, dass $P(Krank | Test)$ über die Formel:

$$P(Krank | Test) = \frac{P(Test | Krank) \cdot P(Krank)}{P(Test)}$$

berechnet werden kann.

Es fehlt jedoch der Wert $P(Test)$, welcher wiederum über die Summe der positiv getesteten kranken und positiv getesteten gesunden Personen berechnet werden kann, was der Formel:

$$P(Test) = P(Test | Krank) \cdot P(Krank) + P(Test | \neg Krank) \cdot P(\neg Krank)$$

entspricht.

Mit einer Krankheitswahrscheinlichkeit von $P(Krank) = 1\%$ ergibt sich für $P(Test)$

$$\begin{aligned} P(Test) &= 0.99 \cdot 0.01 + 0.01 \cdot 0.99 \\ &= 0.0198 \\ &= 1.98\% \end{aligned}$$

und damit für $P(Krank | Test)$

$$\begin{aligned} P(Krank | Test) &= \frac{0.99 \cdot 0.01}{0.0198} \\ &= 0.5 \\ &= 50\% \end{aligned}$$

Für eine Krankheitswahrscheinlichkeit von $P(Krank) = 0.01\%$ ergibt sich für $P(Test)$

$$\begin{aligned} P(Test) &= 0.99 \cdot 0.0001 + 0.01 \cdot 0.9999 \\ &= 0.01 \\ &= 1\% \end{aligned}$$

und damit für $P(Krank \mid Test)$

$$\begin{aligned} P(Krank \mid Test) &= \frac{0.99 \cdot 0.0001}{0.01} \\ &= 0.0099 \\ &= 0.99\% \\ &\approx 1\% \end{aligned}$$

Damit beträgt die Wahrscheinlichkeit, dass eine Person krank ist, unter der Voraussetzung, dass ihr Test positiv ist, bei einer Krankheitswahrscheinlichkeit von 1% etwa 50%.

Die Wahrscheinlichkeit, dass eine Person krank ist, unter der Voraussetzung, dass ihr Test positiv ist, bei einer Krankheitswahrscheinlichkeit von 0.01%, liegt hingegen bei etwa 1%.

Diese Werte stimmen dabei mit den Werten aus der Simulation überein.

This notebook was converted with convert.ploomber.io